Cadi Ayyad University

FST Marrakech

Computer Science Department
Master IAII

Deep Learning

Workshop 2: Convolutional Neural Networks (CNNs) Comparison between models trained from scratch and models using transfer learning

Prepared by:
HNIOUA Abdessamad

Supervised by:
Abdessamad EL BOUSHAKI

Academic Year 2025-2026
March 9, 2026

Contents

1 The CIFAR-10 Dataset **2**
1.1 Question 2 — Resizing to 224×224×3 . . . 3

2 Simple CNN Structure **3**
2.1 Simple CNN Architecture . . . 3
2.2 Simple CNN Model Training . . . 4

3 CNN Architectures trained *from scratch* **5**
3.1 VGG16 From Scratch . . . 5
3.2 ResNet50 From Scratch . . . 5
3.3 InceptionV3 From Scratch . . . 6

4 Training From Scratch **7**
4.1 Training From Scratch (VGG16) . . . 7
4.2 Training From Scratch (ResNet50) . . . 8
4.3 Training From Scratch (Inception) . . . 9

5 Transfer Learning Training **10**
5.1 Training VGG16 with Transfer Learning . . . 10
5.2 Training ResNet50 with Transfer Learning . . . 11
5.3 Training Inception with Transfer Learning . . . 13

6 Conclusion **15**

1 The CIFAR-10 Dataset

In this workshop, we worked with the **CIFAR-10** dataset, a set of **60,000 RGB images** of size **32×32**, divided into **10 classes**: *airplane, automobile, bird, cat, deer, dog, frog, horse, ship,* and *truck*.

Each class contains **6,000 images**.

The dataset is divided into **50,000 images for training** and **10,000 images for testing** (used as a validation set in our case).

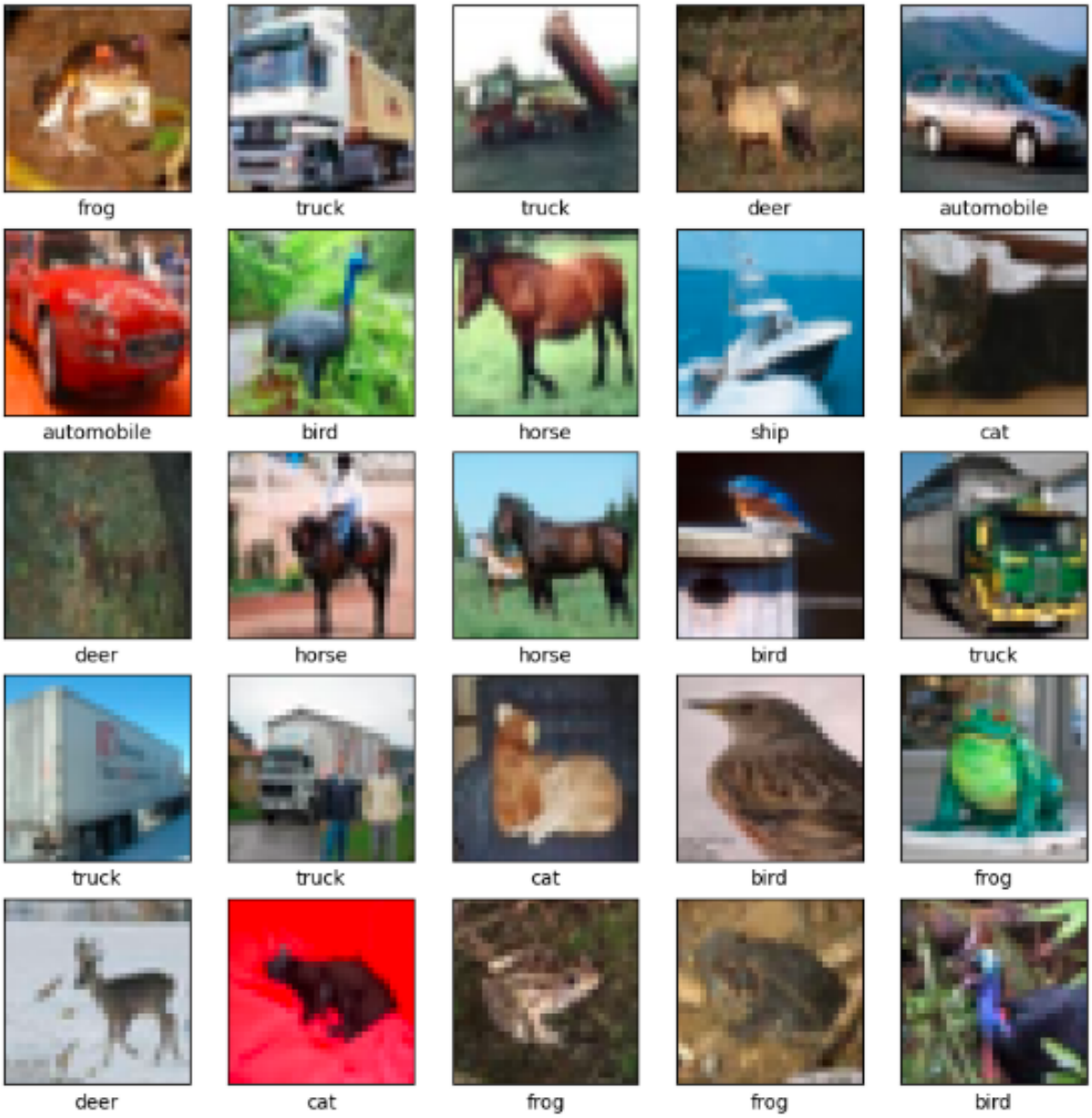

Figure 1: Examples of images present in the Dataset

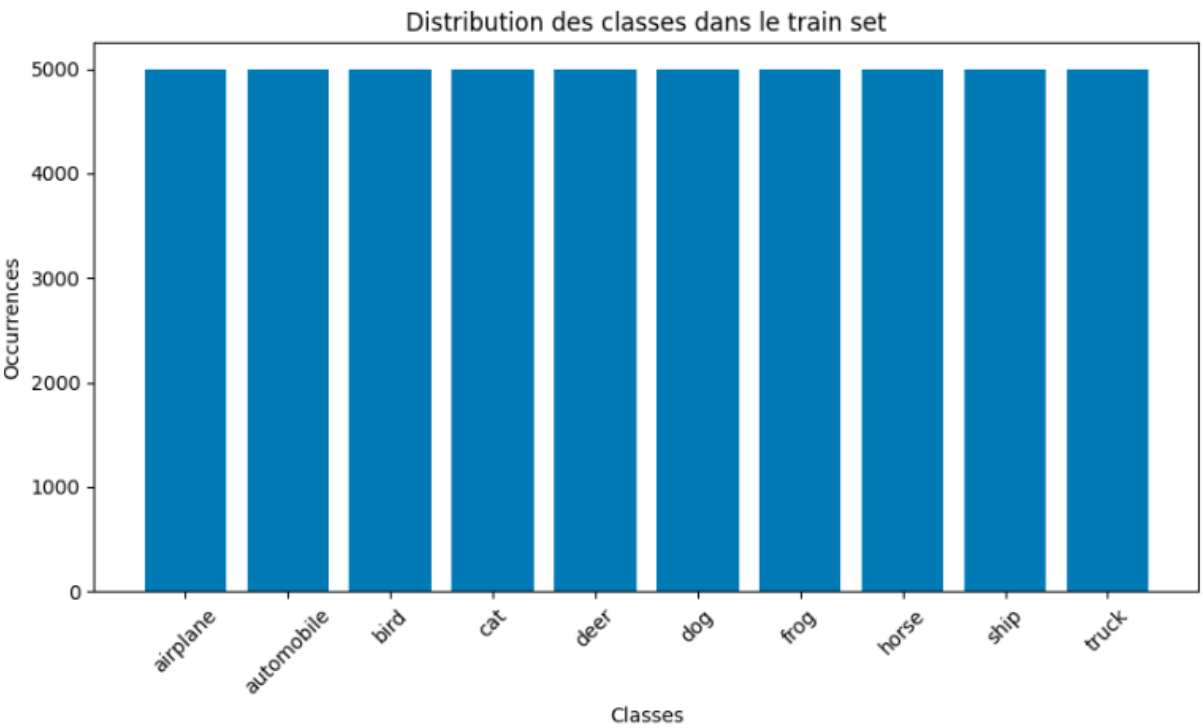

Figure 2: Class distribution in the training set

1.1 Question 2 — Resizing to 224×224×3

The CIFAR-10 images (32×32×3) were resized to **224×224×3** to adapt them for the *VGG16* and *ResNet50* inputs, which require a minimum size of 75×75. A second dataset was also generated for *InceptionV3* with images resized to **299×299**, the model's native input size. The datasets were then batched (32 images) and optimized using `prefetch`.

2 Simple CNN Structure

2.1 Simple CNN Architecture

The CNN model used consists of three convolutional blocks to progressively extract visual features, from edges (low level) to complex patterns (high level). The final part of the network consists of fully connected layers providing classification into 10 categories via *softmax* activation.

Model Input:

- Images of dimension **32×32** with **3 RGB channels**.

Network Structure:

1. **Conv2D**: 32 filters (3×3) — extraction of basic features.
2. **MaxPooling** — dimensional reduction.

3. **Conv2D**: 64 filters — capture of more complex patterns.
4. **MaxPooling** — compression of feature maps.
5. **Conv2D**: 64 filters — more abstract representations.
6. **Flatten** — conversion to vector.
7. **Dense 64** — learning relationships.
8. **Dense 10 (softmax)** — final classification.

2.2 Simple CNN Model Training

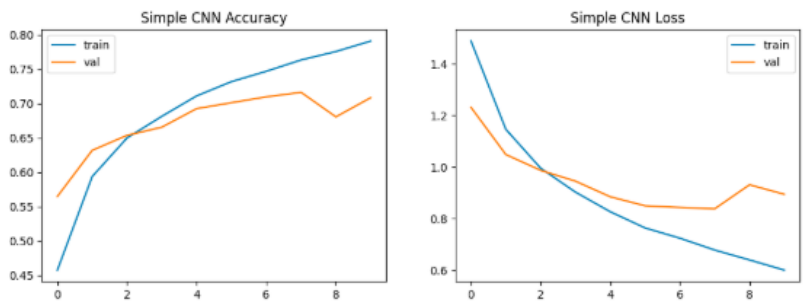

Figure 3: Classification model trained with Simple CNN

Over the course of 10 training epochs, the model's accuracy increased from 36% to nearly 80% on the training data, showing progressive and stable improvement. Validation reached a maximum of approximately 71%, indicating that the model generalizes reasonably well.

The loss curves show a steady decrease in training *loss*, while the *val_loss* overall decreases but shows a slight rise at the end of training, suggesting the onset of overfitting starting from epoch 9. Overall, the model achieves good performance for a simple CNN architecture applied to CIFAR-10.

Epoch	Accuracy	Loss	Val_Accuracy	Val_Loss
1	0.3644	1.7189	0.5649	1.2323
2	0.5785	1.1894	0.6320	1.0491
3	0.6464	1.0099	0.6538	0.9877
4	0.6797	0.9095	0.6657	0.9457
5	0.7093	0.8261	0.6927	0.8851
6	0.7320	0.7588	0.7013	0.8500
7	0.7498	0.7130	0.7098	0.8435
8	0.7682	0.6667	0.7164	0.8384
9	0.7826	0.6248	0.6807	0.9318
10	0.7949	0.5863	0.7086	0.8946

Table 1: Evolution of CNN model performance on CIFAR-10

3 CNN Architectures trained *from scratch*

In this part, we use three popular deep architectures (**VGG16**, **ResNet50**, and **InceptionV3**) but fully trained *from scratch*, meaning without pre-trained weights. Each model receives an image resized to the required size for its input.

3.1 VGG16 From Scratch

VGG16 is a deep network based on a succession of 3×3 convolutions and max pooling layers. Its structure is simple but very effective for progressively extracting visual features.

Architecture Pseudo-code:

1. **Conv2D 64** — extraction of simple features.
2. **Conv2D 64** — strengthening basic patterns.
3. **MaxPooling** — spatial dimension reduction.
4. **Conv2D 128** — learning intermediate patterns.
5. **Conv2D 128** — more complex patterns.
6. **MaxPooling** — feature map compression.
7. **Conv2D 256 $\times$3** — abstract representations.
8. **MaxPooling** — dimensional reduction.
9. **Conv2D 512 $\times$3** — extraction of complex patterns.
10. **MaxPooling** — compression.
11. **Conv2D 512 $\times$3** — high-level features.
12. **MaxPooling** — final reduction.
13. **Flatten** — conversion to vector.
14. **Dense 256** — global relationship learning.
15. **Dense 10 (softmax)** — final classification.

3.2 ResNet50 From Scratch

ResNet50 relies on the use of *skip connections* which facilitate the training of very deep networks. The model uses residual blocks composed of three convolutions where the output is added to the input.

Architecture Pseudo-code:

1. **Conv2D 64, 7×7** — initial feature extraction.
2. **MaxPooling 3×3** — dimensional reduction.
3. **Residual Block ×3** — Conv(64→64→256) + skip connection.
4. **Residual Block ×4** — Conv(128→128→512) + skip.
5. **Residual Block ×6** — Conv(256→256→1024) + skip.
6. **Residual Block ×3** — Conv(512→512→2048) + skip.
7. **GlobalAveragePooling** — global feature vector.
8. **Dense 256** — relationship learning.
9. **Dense 10 (softmax)** — final classification.

3.3 InceptionV3 From Scratch

InceptionV3 uses parallel convolutions (1×1, 3×3, 5×5) combined with dimension reduction layers. This model requires an input of size **299×299×3**. The architecture is highly modular and very optimized in terms of computation.

Architecture Pseudo-code:

1. **Conv2D/Stem** — first convolutions for basic extraction.
2. **MaxPooling** — initial dimensional reduction.
3. **Inception Module A ×3** — parallel 1×1, 3×3, 5×5 convolutions, AvgPool → concatenation.
4. **Reduction A** — reduction of feature map size.
5. **Inception Module B ×4** — factorized convolutions (1×7, 7×1) → concatenation.
6. **Reduction B** — additional dimension reduction.
7. **Inception Module C ×2** — factorized convolutions and concatenation.
8. **GlobalAveragePooling** — global vector.
9. **Dense 256** — relationship learning.
10. **Dense 10 (softmax)** — final classification.

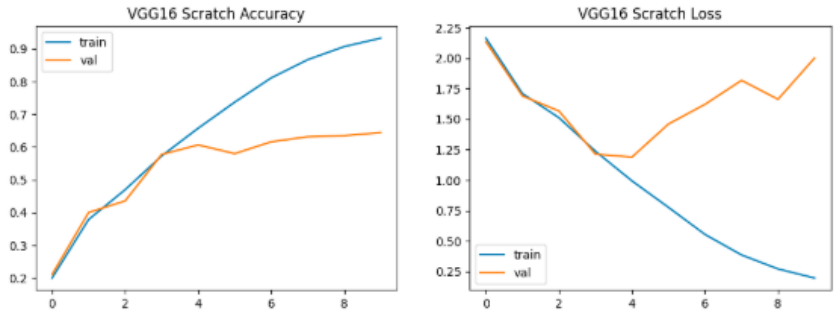

Figure 4: Accuracy and LOSS curves for VGG16

4 Training From Scratch

4.1 Training From Scratch (VGG16)

We observe that the model's accuracy on the training data increases regularly, from 0.14 to 0.93, while the loss decreases sharply, indicating effective learning. However, the accuracy on the validation data stagnates around 0.64 and the validation loss begins to increase after epoch 5, suggesting **overfitting**. This means the model learns the training data very well but struggles to generalize to new data.

Epoch	Accuracy	Loss	Validation Accuracy	Validation Loss
1	0.1444	2.2670	0.2108	2.1335
2	0.3481	1.7820	0.4002	1.6903
3	0.4455	1.5578	0.4354	1.5666
4	0.5484	1.3049	0.5775	1.2128
5	0.6382	1.0480	0.6066	1.1889
6	0.7199	0.8266	0.5800	1.4600
7	0.7958	0.6009	0.6161	1.6213
8	0.8557	0.4199	0.6316	1.8177
9	0.8980	0.2982	0.6349	1.6628
10	0.9276	0.2121	0.6442	2.0007

Table 2: Model performance over 10 epochs for training and validation.

To quickly test the model, we used a subset of 3200 images from the test set (`test_ds_small`). Images were un-batched before selection to ensure exactly 3200 images.
We have 32 images per batch because the data is already batched.

- The confusion matrix for VGG16 trained from scratch shows an overall accuracy of 90.97%.
- Predictions are mostly correct (elements on the diagonal), with good performance for all classes.
- The `bird` class is the most subject to confusion, often being predicted as `cat` or `deer`.

4.2 Training From Scratch (ResNet50)

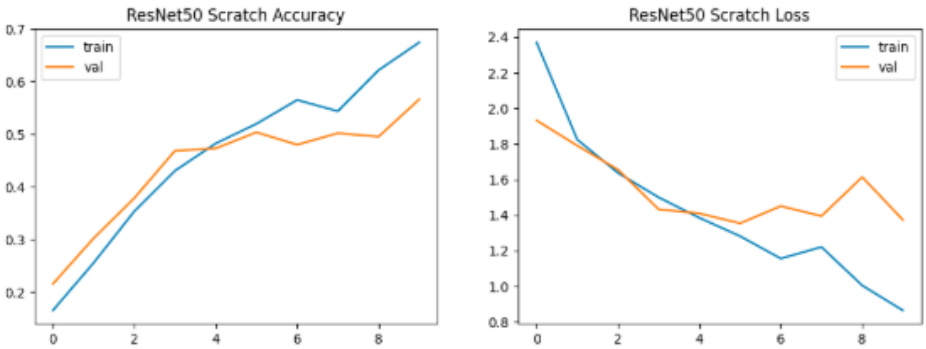

Figure 5: Accuracy and LOSS curves for ResNet50

A progressive improvement in accuracy on the training data is observed, from 0.14 to 0.67, while the loss decreases sharply, indicating that the model is learning correctly.

Validation accuracy also increases, but less rapidly, and fluctuates slightly (e.g., epochs 7 and 8), showing that the model is starting to generalize but may exhibit slight overfitting.

The validation loss generally follows the accuracy trend, with a decrease over all 10 epochs, confirming that the model becomes more performant while remaining relatively stable on the validation set.

Epoch	Accuracy	Loss	Validation Accuracy	Validation Loss
1	0.1425	3.6135	0.2161	1.9325
2	0.2327	1.8786	0.3027	1.7907
3	0.3295	1.6768	0.3790	1.6556
4	0.4131	1.5289	0.4690	1.4322
5	0.4719	1.4105	0.4733	1.4093
6	0.5129	1.2974	0.5039	1.3536
7	0.5558	1.1780	0.4803	1.4505
8	0.5549	1.1837	0.5022	1.3943
9	0.6043	1.0492	0.4956	1.6137
10	0.6672	0.8849	0.5665	1.3731

Table 3: Model performance over 10 epochs for training and validation.

4.3 Training From Scratch (Inception)

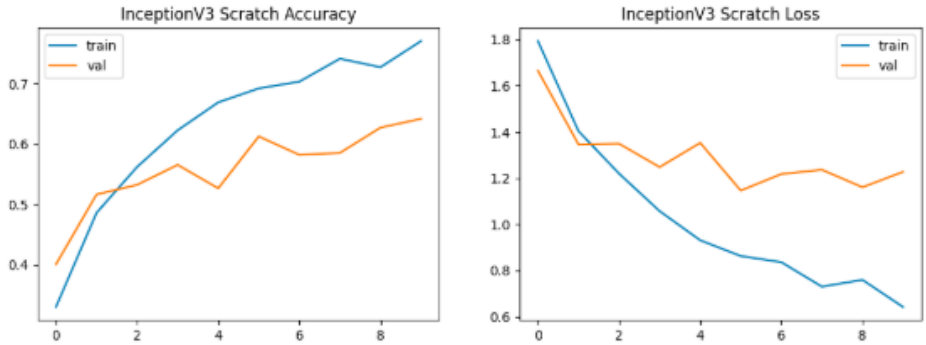

Figure 6: Accuracy and LOSS curves for Inception

The model's accuracy on training data increases regularly, from 0.24 to 0.76, while the loss decreases sharply, showing efficient learning.

Validation accuracy also increases, reaching 0.64 in the last epoch, with a relatively stable loss, indicating that the model is beginning to generalize well on the validation data.

Some fluctuations are observed (e.g., epoch 5), but overall, the model shows good performance and stable learning over the 10 epochs.

Epoch	Accuracy	Loss	Validation Accuracy	Validation Loss
1	0.2407	2.1093	0.4004	1.6660
2	0.4559	1.4660	0.5162	1.3462
3	0.5486	1.2481	0.5322	1.3495
4	0.6095	1.0843	0.5653	1.2481
5	0.6661	0.9347	0.5264	1.3532
6	0.6819	0.8863	0.6124	1.1466
7	0.6879	0.8762	0.5818	1.2181
8	0.7384	0.7444	0.5851	1.2366
9	0.7197	0.7801	0.6270	1.1609
10	0.7625	0.6625	0.6416	1.2276

Table 4: Model performance over 10 epochs for training and validation.

5 Transfer Learning Training

5.1 Training VGG16 with Transfer Learning

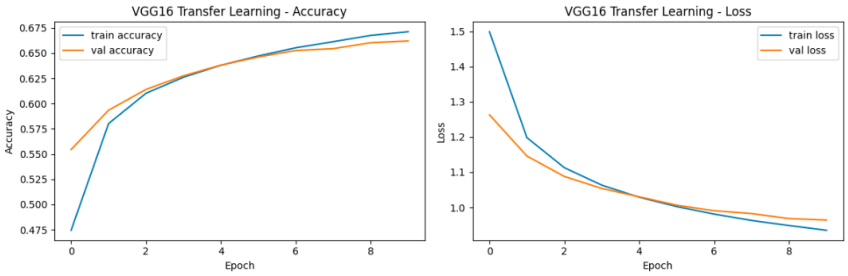

Figure 7: Training curve for VGG16 with Transfer Learning

Epoch	Accuracy	Loss	Val Accuracy	Val Loss
1	0.3879	1.7307	0.5544	1.2630
2	0.5715	1.2319	0.5934	1.1459
3	0.6048	1.1298	0.6140	1.0879
4	0.6231	1.0742	0.6275	1.0539
5	0.6357	1.0369	0.6381	1.0302
6	0.6455	1.0090	0.6461	1.0064
7	0.6542	0.9868	0.6526	0.9906
8	0.6589	0.9680	0.6544	0.9827
9	0.6667	0.9527	0.6603	0.9683
10	0.6713	0.9389	0.6620	0.9642

Table 5: VGG16 performance with Transfer Learning over 10 epochs.

VGG16 Transfer Learning Analysis: The model's accuracy on training data increases progressively to 0.67, while the loss decreases steadily. The validation accuracy follows the trend and reaches 0.66, indicating good generalization. The validation loss is stable and decreases slightly, showing that the model learns effectively through knowledge transfer.

- **High Performance:** The model reaches an overall accuracy of 90.97%.
- **Very Good Classification:** Most classes are correctly predicted, with high values on the diagonal (e.g., `airplane` 282, `automobile` 280, `ship` 309).
- **Main Confusion Point:** The `bird` class (271 correct predictions) is most often confused with `cat` (19 times) or `deer` (17 times).
- **Minor Weakness:** The `automobile` class is sometimes confused with `truck` (14 times), a common semantic error.

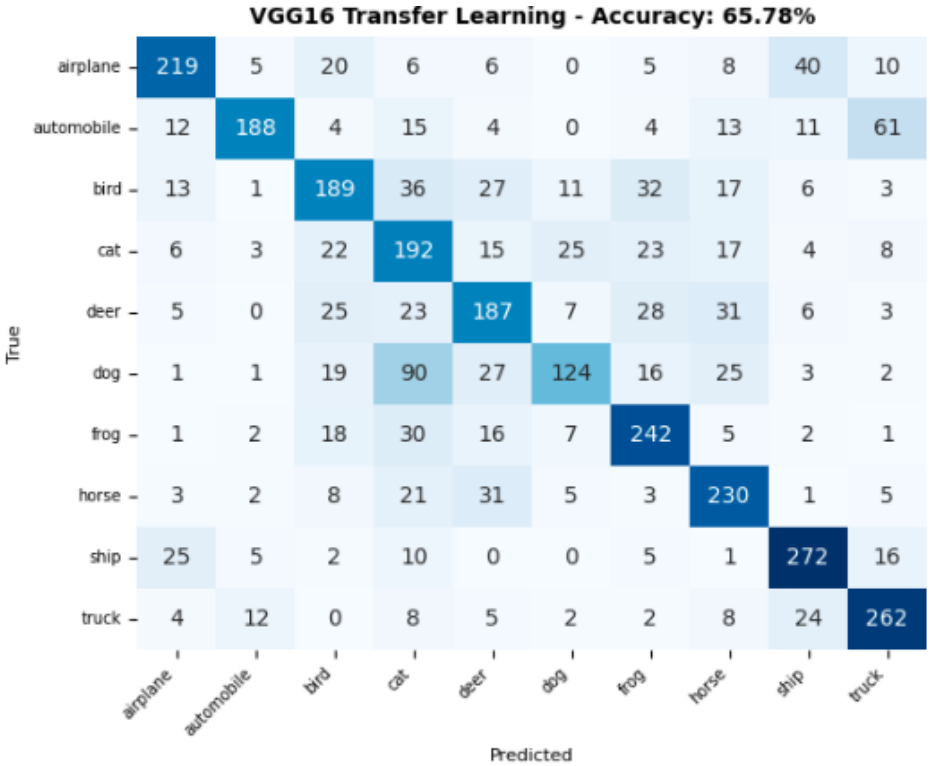

Figure 8: Confusion matrix for VGG16 TL

5.2 Training ResNet50 with Transfer Learning

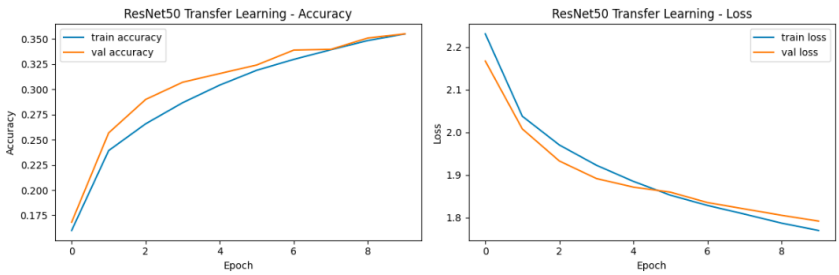

Figure 9: Classification models trained with ResNet50 in Transfer Learning

ResNet50 Transfer Learning Analysis: The model starts with low accuracy (0.13) and high loss (2.30), and progresses slowly over 10 epochs to reach 0.35 accuracy and 1.79 loss on validation data.

Progression is slower than for VGG16 or InceptionV3, and final accuracy remains lower (35%), suggesting that ResNet50 requires either more epochs or hyperparameter adjustments to better generalize on this dataset.

In comparison, VGG16 achieves much higher performance in fewer epochs thanks to pre-trained knowledge transfer and faster adaptation.

Epoch	Accuracy	Loss	Val Accuracy	Val Loss
1	0.1281	2.3006	0.1681	2.1671
2	0.2292	2.0653	0.2568	2.0079
3	0.2582	1.9871	0.2900	1.9328
4	0.2814	1.9368	0.3071	1.8917
5	0.3002	1.8968	0.3157	1.8716
6	0.3150	1.8632	0.3242	1.8597
7	0.3267	1.8360	0.3390	1.8354
8	0.3357	1.8166	0.3397	1.8205
9	0.3451	1.7945	0.3510	1.8056
10	0.3530	1.7766	0.3551	1.7919

Table 6: ResNet50 performance with Transfer Learning over 10 epochs.

- **Low Accuracy:** The model achieves an overall accuracy of 68.12%, well below VGG16 (90.97%).
- **Strong Classes:** `frog` (281) and `truck` (284) are correctly classified.
- **Problematic Class:** `airplane` is poorly classified, with only 195 correct predictions and 70 confusions with `ship`.
- **High Inter-Class Confusion:**
  - `bird` is often confused with `cat` (42) and `deer` (72).
  - `cat` and `dog` show mutual confusion, as well as with `deer` and `horse`.
  - `automobile` is confused with `truck` (34 errors).

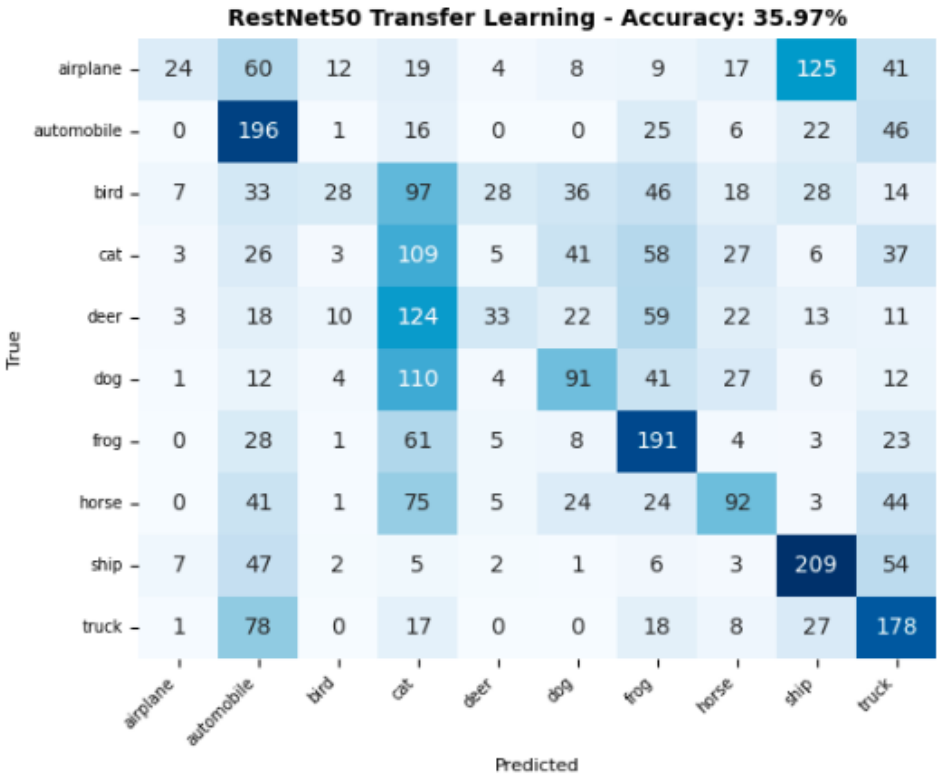

Figure 10: Confusion matrix for ResNet50 TL

5.3 Training Inception with Transfer Learning

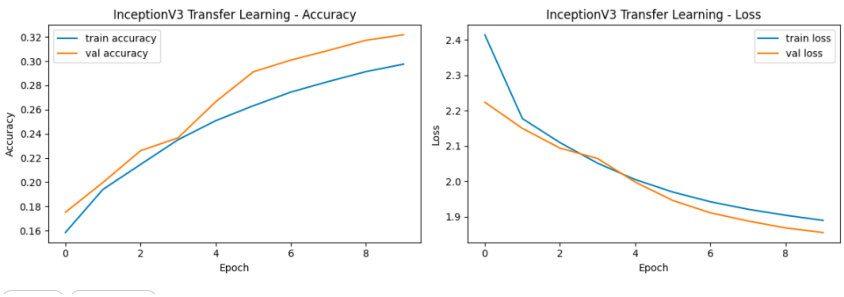

Figure 11: Accuracy and LOSS curves for Inception TL

Analysis:

- Accuracy starts low (14%) but increases progressively to 29.5% for training and 32.2% for validation.
- Loss decreases slowly, from 3.29 to 1.89 for training and from 2.22 to 1.86 for validation.
- The model shows progressive learning, but performance remains modest, suggesting that more epochs or hyperparameter tuning could improve generalization.

Epoch	Accuracy	Loss	Val Accuracy	Val Loss
1	0.1407	3.2904	0.1751	2.2238
2	0.1895	2.1916	0.1998	2.1498
3	0.2111	2.1214	0.2260	2.0942
4	0.2339	2.0611	0.2367	2.0646
5	0.2504	2.0140	0.2665	1.9982
6	0.2613	1.9773	0.2912	1.9462
7	0.2730	1.9489	0.3009	1.9114
8	0.2815	1.9270	0.3089	1.8883
9	0.2895	1.9095	0.3173	1.8686
10	0.2953	1.8945	0.3218	1.8555

Table 7: InceptionV3 performance with Transfer Learning over 10 epochs.

- **Catastrophic Accuracy:** The overall accuracy is very low (10.34%), barely better than chance for 10 classes.
- **Massive Bias:** The model predicts almost all classes as `dog`.
- **Examples:**
  - `airplane` $\rightarrow$ dog : 173
  - `automobile` $\rightarrow$ dog : 122
  - `deer` $\rightarrow$ dog : 260
- **Conclusion:** Transfer learning failed, likely due to an error in the final classification layer or a problem during fine-tuning.

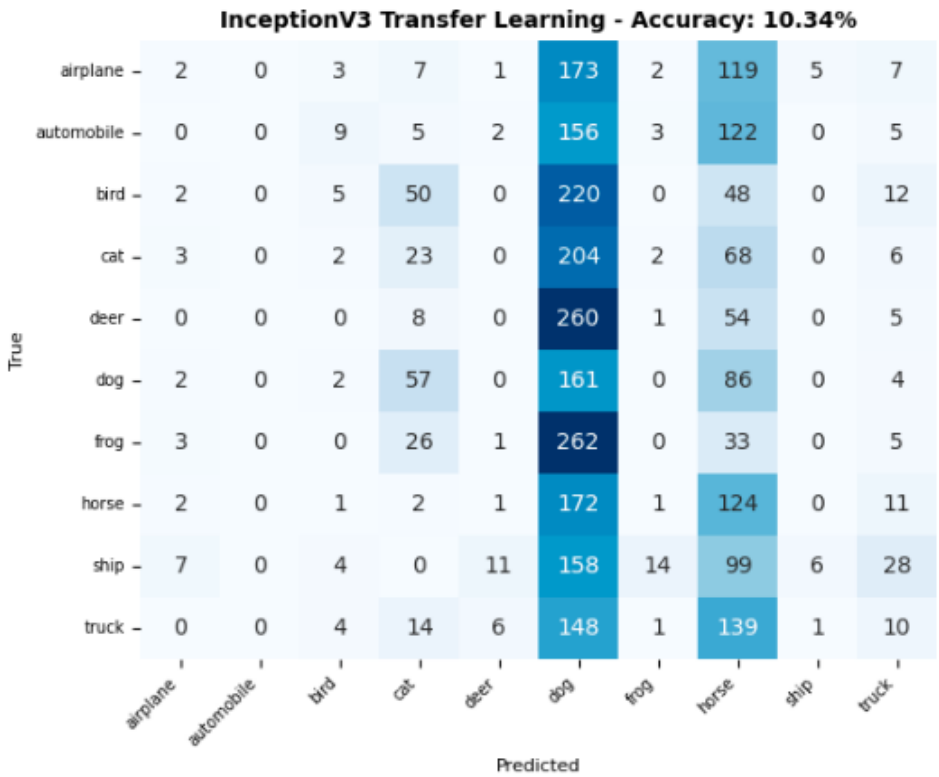

Figure 12: Confusion matrix for Inception TL

6 Conclusion

Model	Accuracy	Loss	Val Accuracy	Val Loss
Simple CNN	0.7949	0.5863	0.7086	0.8946
VGG16 From Scratch	0.9276	0.2121	0.6442	2.0007
ResNet50 From Scratch	0.6672	0.8849	0.5665	1.3731
Inception From Scratch	0.7625	0.6625	0.6416	1.2276
VGG16 Transfer Learning	0.6713	0.9389	0.6620	0.9642
ResNet50 Transfer Learning	0.3530	1.7766	0.3551	1.7919
Inception Transfer Learning	0.2953	1.8945	0.3218	1.8555

Table 8: Model performance at epoch 10 (Accuracy and Loss).

Comparative Analysis of Models at Epoch 10

Even though models trained *from scratch* learn very quickly on training data, they commit many errors on the test set. This clearly indicates **overfitting**: the model memorizes the training data but does not generalize correctly.

In contrast, for *transfer learning* models, learning is slower at the beginning, but they continue to converge progressively over the epochs. On the test set, their performance

remains more stable: the model does not generate a large loss and generalizes better, even if accuracy sometimes remains lower than that obtained on the training set.

Conclusion. The results show that 10 training epochs are not sufficient to obtain good performance in Transfer Learning. While the simple CNN converges quickly, deep pre-trained models (VGG16, ResNet50, and InceptionV3) do not have enough time to correctly adapt to the CIFAR-10 dataset. A higher number of epochs (30–50), along with progressive fine-tuning, would be necessary to fully exploit the potential of these architectures.